

Implant, Conceal, Resurrect: Installing an Exact Circuit in a Real Language Model and Its Fate Under Continued Training

Gunner Levi Howe
gunnerlevihowe@gmail.com

July 2026

Abstract

We construct a verified-exact algorithmic circuit — a one-bit toggle automaton — inside a single head of a *trained* linear-RNN language model, and study what continued training does to it. Two construction facts frame the study. (i) The dense residual stream of a 30M-parameter tied-embedding model has no spare capacity (least-active residual dimension standard deviation 0.717 vs. median 0.87), so an in-place behavioral readout collides with the model’s own RMSNorm and destroys language quality (perplexity rises by $\approx$ fourteen orders of magnitude). (ii) Adding one inert head (widening $d=512 \rightarrow 576$, $H=8 \rightarrow 9$ by zero-padding, verified identity to -0.13% perplexity) yields a clean substrate: the implanted circuit reads out behaviorally at 1.00 accuracy through toggle depth 64 at -0.098% perplexity cost — an exact skill installed in a real LM, for free. We then continue training under a pre-registered grid (arms {all-trainable, freeze-implant} $\times$ exercise mixture {0, 10, 50%}). The central result: under all-trainable pure-LM training with no exercise, behavioral toggle decays to chance ($1.00 \rightarrow 0.52$) while the internal circuit remains exact at the end of training (1.0), and re-pinning only the readout interface restores full behavior (1.0). The mechanism is *interface drift*: continued training moves the tied-embedding readout, suppressing behavioral *expression* while the *computation* persists and is weight-recoverable. A 10% exercise trickle prevents the drift entirely (behavior stays 1.00); freezing the circuit’s weights but not its interface is *counterproductive* (internal circuit erodes to 0.0, unrecoverable) because frozen weights cannot co-adapt to the drifting input; and heavy (50%) exercise diffuses the skill beyond the implanted head. We position this precisely: it is a ground-truth demonstration, at toy scale, of a phenomenon the unlearning-robustness literature studies behaviorally — suppression is not removal — with the honest caveats that the circuit is constructed (not learned), the model is a linear RNN (not a transformer), the skill is trivial, and recovery is white-box. All predictions and kills were pre-registered before the runs; every number regenerates from committed artifacts.

1 Introduction

Two capabilities the field would like but cannot yet reliably perform: *installing* a specific computation into a trained model, and *knowing* whether subsequent training removed a computation or merely hid its expression. The second is a live safety question — unlearning and safety fine-tuning are only useful if they *remove* a capability rather than suppressing its surface behavior [8] — but it is hard to answer without a ground-truth circuit: when performance drops, probing cannot tell erasure from concealment.

This paper builds the setting where the question is decidable. We take the verified-exact state-tracking constructions of our companion work [4] — solutions written into DeltaProduct-style Householder linear RNNs and provably correct at initialization — and use them as *implants*: an exact circuit inserted by hand into a trained language model, whose integrity after any amount of further training can be read against a known ceiling; the toy-scale study of implant fate is our companion [6], which this paper takes to a real LM. Because the delta rule performs one reflection per token per head, the implantable target is the reflection-length-one task — a toggle (lamp on/off) automaton, which is also the trained model’s own hardest reliably-learned skill in Howe [5]. We construct it, verify it reads out exactly, continue training

under a pre-registered grid, and measure both its behavioral expression (via the model’s logits) and its internal computation (the implanted head’s state, read directly).

What is and is not new. Compiling exact algorithms into transformer weights is established — Tracr [7] compiles RASP programs into fresh transformers, and InterpBench [3] trains ground-truth circuits into semi-synthetic models. Our construction is the linear-RNN analog, and is *not* the contribution. The contribution is what happens *after* insertion into a *trained* model under *continued training*: the concealment / recovery dynamics, their interface-drift mechanism, and the exercise/freeze dependence. Likewise, “suppression is not removal” is a known concern in unlearning [8] and edit durability [1]; our contribution there is a ground-truth exact-circuit demonstration and mechanism, not the concept.

Contributions.

1. **An exact circuit installed in a real LM, measured (§3):** the dense residual stream forbids a cheap in-place behavioral readout (a construction obstacle we quantify); one inert added head fixes it, giving exact behavioral readout at -0.098% perplexity.
2. **Concealment and recovery under continued training (§4):** pre-registered; unexercised pure-LM training suppresses behavior to chance while the circuit stays internally exact and readout-recoverable.
3. **The interface-drift mechanism (§5):** behavior dies because the tied-embedding readout drifts, not because the computation is destroyed; exercise pins the interface, and freezing the circuit alone is counterproductive.
4. **Honest scope and safety framing (§6–§7):** what a toy, constructed, white-box result does and does not imply for frontier unlearning.

2 Setup

Model. The trained substrate is the 30M-parameter DeltaNet-class LM of Howe [5] (the arm-A checkpoint; $d=512$, 8 layers, 8 heads, tied embeddings, delta rule $S_t=(I - \beta_t k_t k_t^\top)S_{t-1} + g_t \beta_t k_t v_t^\top$, $o_t=S_t^\top q_t$), trained on TinyStories [2] to perplexity 3.08. **Implant.** Toggle has reflection length one, so it fits a single head: the lamp lives at one state entry, a content token sets it, a flip token negates it via the $\beta=2$ erase term, a check token reads it. Exercise uses four dedicated vocabulary tokens (on/off/flip/check); behavioral toggle is scored from the model’s next-token logits, internal toggle from the implanted head’s output at the check position read directly. All predictions/kills were frozen in-repository before any continued-training run.

3 Installing the circuit

The dense-stream obstacle. The toy constructions of Howe [4] live in a clean dedicated-dimension subspace. A trained tied-embedding LM has none: measured over TinyStories, the least-active residual dimension has standard deviation 0.717 against a median of 0.87 (max 2.671) — every dimension carries language. Consequently, donating a head is cheap but head-specific (Fig. 1a: block-0 head 1 costs 0.3% perplexity, head 0 6.2%, head 6 a catastrophic 431.9% — it does induction-like work), yet routing the circuit’s answer to the logits *in place* collides with the block’s output RMSNorm, which amplifies the donated head’s near-zero output on quiet real tokens: the behavioral readout works (1.00 toggle) but drives perplexity to $\approx 10^{14}$. In a dense residual stream, an exact circuit can be installed and computed but cannot be cheaply wired to the output.

The widened substrate. Adding one inert 64-dimensional head ($d=512 \rightarrow 576$, $H=8 \rightarrow 9$) by zero-padding every weight matrix is verified identity: the widened-but-empty model matches the trained model to -0.13% perplexity. Implanting the toggle circuit into that fresh head, with the answer on a fresh residual dimension, is then clean — the new dimensions are *exactly* zero for real tokens, so the query is exactly zero on real text and the RMSNorm has nothing to amplify. The result (Fig. 1b): behavioral toggle 1.00 through

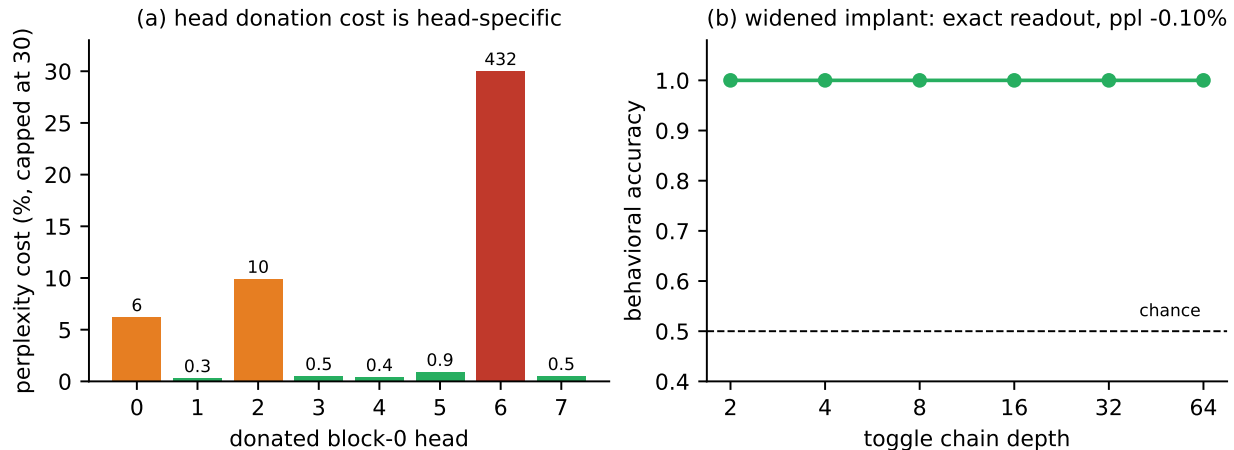


Figure 1: **(a)** Perplexity cost of donating each block-0 head (capped at 30%; head 6 is 431.9%): donation is cheap but head-specific. **(b)** In the widened substrate the implanted circuit reads out behaviorally at 1.00 through depth 64, at -0.098% perplexity.

arm	p	beh. start	beh. end	internal end	resurrection	ppl end
none	0%	1.00	0.52	1.00	1.00	3.41
none	10%	1.00	1.00	1.00	1.00	3.47
none	50%	1.00	1.00	0.45	1.00	3.72
freeze	0%	1.00	0.00	0.00	0.00	3.42
freeze	10%	1.00	1.00	0.45	1.00	3.49
freeze	50%	1.00	1.00	0.76	1.00	3.74

Table 1: Continued-training battery (widened substrate, seed 0, 3000 steps; toggle chance 0.5; base perplexity 3.08). “Beh.” = behavioral (logits); “internal” = the implanted head’s own computation; “resurrection” = behavioral toggle after re-pinning the readout interface.

depth 64 (well beyond any training length) at -0.098% perplexity cost. An exact algorithmic skill now lives in a real language model and the model emits it, at no language cost.

4 Fate under continued training

We continue training the implanted widened model for 3000 steps under a pre-registered 2×3 grid: arm $\in \{\text{none (all weights trainable), freeze-implant (the head’s implant weights frozen)}\} \times$ exercise mixture $p \in \{0, 10, 50\%\}$ of dedicated-token toggle sequences vs. pure TinyStories. We log behavioral and internal toggle and perplexity every 300 steps, and after training re-pin the readout interface (the answer-routing weight and the on/off embedding rows) and re-measure behavior (the resurrection probe). Table 1 and Figs. 2–3 give the results.

Pre-registered verdicts.

- **IL1 (unexercised erosion): confirmed.** none/0% behavioral toggle decays $1.00 \rightarrow 0.52$ (≤ 0.60).
- **IL2 (exercise protects): confirmed.** none end-behavior is nondecreasing in exercise (0.52, 1.0, 1.0); a 10% trickle holds it at 1.00 throughout.
- **IL3 (freeze preserves): not confirmed — and the failure is a finding.** freeze/0% dies to 0.0 and does not recover (0.0), *worse* than the all-trainable none/0% (§5).
- **IL4 (resurrection): confirmed where applicable.** Every cell whose behavior decayed while the internal circuit stayed intact recovers to 1.00 by re-pinning the readout (none/0%: behavioral 0.52,

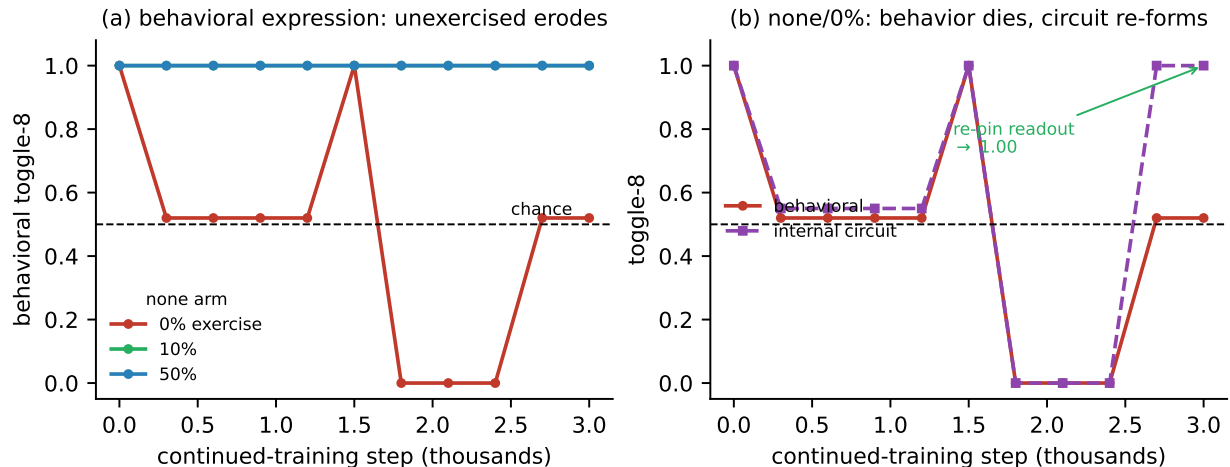


Figure 2: **(a)** Behavioral toggle over continued training (none arm): unexercised (0%) erodes to chance; a 10% or 50% exercise trickle holds it at 1.00. **(b)** none/0%: behavioral expression collapses while the internal circuit — knocked around during training, but exact at the end — persists; re-pinning the readout restores behavior.

internal 1.0, resurrection 1.0). The one genuinely eroded cell (freeze/0%: internal 0.0) correctly does not recover (0.0): the internal read separates concealed from eroded.

- **Kills.** KL2 (implant unpreservable everywhere) does not fire (none/10% fully preserves). KL3 (exercise collapses perplexity $> +10\%$) technically fires at $p=50\%$ ($+20.6\%$) but is confounded: the no-exercise none/0% cell already drifts $+10.7\%$ from continued training alone, so most of the rise is generic optimization drift, not exercise (§7).

5 The mechanism: interface drift

Why does an unexercised implant lose its behavior but not its computation? The vulnerable object is the *interface*, not the circuit. The dedicated tokens’ embeddings and the answer-readout weights are tied into the model’s shared embedding and receive gradients on every step (the tied embedding is also the output head), so continued training drifts them even when the toggle task never appears in the data. Behavioral toggle therefore decays (Fig. 2b) while the implanted head’s *computation*, read directly, survives.

Two arms confirm the mechanism. **Exercise pins the interface:** at $p \geq 10\%$ the dedicated tokens appear in training, their embeddings stay put, and behavior never drifts (IL2). **Freezing the circuit is counterproductive:** in none/0% the trainable circuit *co-adapts* to the drifting input and its internal integrity re-forms to exact by the end (1.0); in freeze/0% the frozen circuit cannot compensate for the drifting input embeddings, so its internal computation breaks (0.0) and is unrecoverable. Preserving an implant thus requires pinning its *input/output interface*, not its weights — the opposite of the naive intervention.

A third, unpredicted phenomenon: at heavy exercise ($p=50\%$) behavioral toggle stays 1.00 while the *single-head* internal read drops (0.45). Heavy practice diffuses the skill beyond the implanted head into redundant routes, so a one-head read no longer reflects behavior. The implant seeds a capability the surrounding model then distributes.

6 Discussion

The clean statement of the result: continued training can drive a deliberately-installed capability’s behavioral *expression* to chance while its underlying *computation* persists intact and is recoverable by re-aligning the readout — and a ground-truth internal read distinguishes this concealment from genuine erasure, which

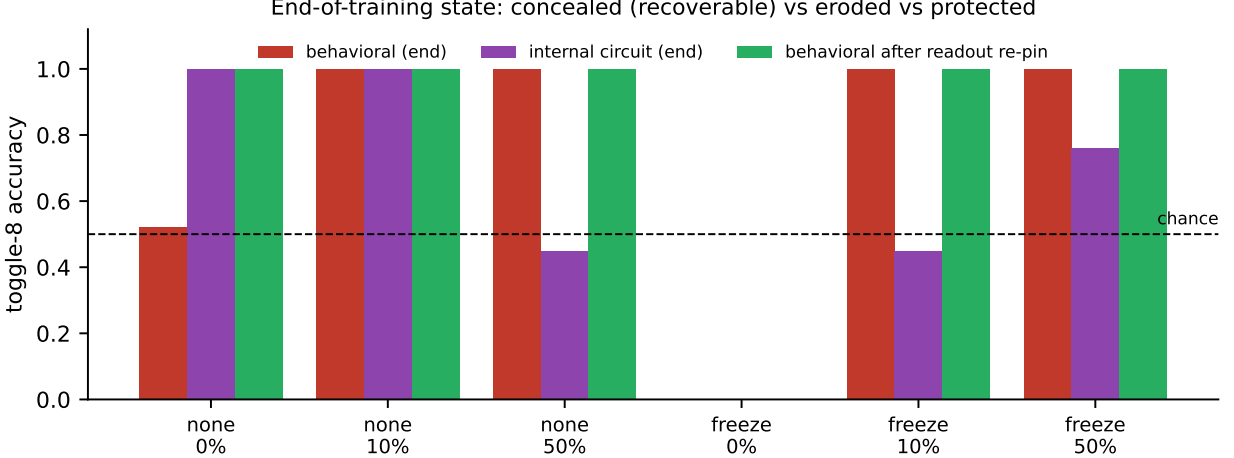


Figure 3: End-of-training state per cell: behavioral toggle, internal-circuit toggle, and behavioral toggle after re-pinning the readout. Concealed-but-recoverable (none/0%: behavior at chance, circuit exact, recovers), eroded (freeze/0%: all at chance, no recovery), and protected (exercised cells) separate cleanly; the internal read is what distinguishes concealment from erasure.

the same battery also produces. This is the mechanistic, provable version of the unlearning-robustness observation that suppression need not be removal [8]: here we can *prove* the mechanism survived because we built it exactly, and we identify *why* (interface drift) and *what prevents preservation* (freezing weights without the interface).

We are deliberately careful about reach. The safety-relevant reading — “training may hide rather than remove capabilities” — is supported here only in a specific, constructed, white-box setting, and the recovery mechanism is interface-specific. What the result contributes to that conversation is a ground-truth existence proof and a named mechanism, not evidence that natural capabilities in frontier models behave this way. Section 7 states the gaps that would have to close before the reading generalizes.

7 Limitations and honest scope

Constructed, not learned. The implant is a hand-built exact circuit; a naturally-trained capability may drift differently. This is the single largest gap to any frontier claim. **Linear RNN, not transformer.** The clean single-head delta-rule construction and the widening trick are architecture-specific; the transformer analog (Tracr-style) is a different, harder construction. **Trivial skill.** Toggle is one bit ($\mathbb{Z}_2$); it is not a capability anyone would care to hide, and higher-reflection-length skills need multi-head constructions (open). **White-box recovery.** “Resurrection” re-pins weights we saved; it is not an adversary recovering a suppressed capability from a released checkpoint. **Dedicated-token protocol.** The skill is exercised through reserved tokens, not natural language; natural-language triggers are untested. **Single seed; perplexity drift.** One seed; and the continued-training setup drifts perplexity +10.7% even at zero exercise (learning rate not tuned for minimal-drift continuation), which does not affect the within-run toggle-integrity findings (all relative to each run’s own start) but means the absolute perplexity numbers are not a clean cost measurement. **The freeze arm’s confound is reported as a finding,** not corrected. None of these is hidden; together they place the work as a toy-scale mechanistic study, and the path to relevance is to close at least two of constructed→learned, RNN→transformer, white-box→realistic-recovery, trivial→meaningful.

8 Reproducibility

All code, the pre-registration (frozen before the battery), the 6 cell artifacts, the fact-capture script (`capture_facts.py`, which regenerates every construction/feasibility number into `facts.json`), and this paper’s number/figure pipeline are published in the `lmiimplant/` directory of <https://github.com/gunnerhowe/automaton-underneath>. Every number here is a macro generated by `gen_numbers.py` from `facts.json` and `runs_wide/*.json`; `verify_regen.py` re-derives them and byte-compares before release. Compute: ≈ 2.5 GPU-hours (battery) plus construction probes on one RTX 3080.

References

- [1] Yinjie Cheng, Paul Youssef, Christin Seifert, Jörg Schlötterer, and Zhixue Zhao. Can fine-tuning erase your edits? on the fragile coexistence of knowledge editing and adaptation. *arXiv preprint arXiv:2511.05852*, 2025.
- [2] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.
- [3] Rohan Gupta, Iván Arcuschin, Thomas Kwa, and Adrià Garriga-Alonso. InterpBench: Semi-synthetic transformers for evaluating mechanistic interpretability techniques. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2407.14494.
- [4] Gunner Levi Howe. The automaton underneath: The additive input pathway is a parasitic attractor for state tracking in Householder linear RNNs, 2026. Manuscript; paper, pre-registrations, and 202 run artifacts at <https://github.com/gunnerhowe/automaton-underneath>.
- [5] Gunner Levi Howe. The gate learns hygiene: The training economics of the write path in linear-RNN language models, 2026. Manuscript; artifacts at <https://github.com/gunnerhowe/automaton-underneath> (twogap/).
- [6] Gunner Levi Howe. Implant, conceal, resist: The fate of inserted skills under continued training in linear RNNs, 2026. Manuscript; toy-scale companion, artifacts at <https://github.com/gunnerhowe/automaton-underneath> (implant/).
- [7] David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2301.05062.
- [8] Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. Eight methods to evaluate robust unlearning in LLMs. *arXiv preprint arXiv:2402.16835*, 2024.